Transformers, BERT and GPT

A friendly, story-style guide to the architecture behind modern language models

> **How to read this guide.** Every section starts with a small story or picture, so you get the *feel* first. Technical details (layers, sizes, training data) come after, and the numbers come from the original research papers listed at the end.

What is inside

- **1.** The problem before Transformers
- **2.** The Transformer: the big picture
- **3.** Self-attention: the heart of it all
- **4.** The supporting parts: positions, feed-forward layers, residuals
- **5.** BERT: the reader
- **6.** GPT: the writer
- **7.** BERT vs GPT side by side
- **8.** Why this matters for RAG, glossary and references

1. The Problem Before Transformers

Imagine reading a long story, but you are only allowed to read **one word at a time**, and you must remember everything in a tiny backpack that holds just a little. By the time you reach the last page, the first page is blurry.

That is how older language models worked. Recurrent Neural Networks (RNNs) and their improved versions, LSTMs and GRUs, read text word by word, passing a small memory along. This caused two big headaches:

- **Forgetting:** information from far-away words faded as the sentence got longer.
- **Slowness:** because each word had to wait for the previous one, the work could not be split across many processors at once.

In 2017, a team at Google published a paper called *Attention Is All You Need* (Vaswani et al.). Their idea: drop the one-word-at-a-time reading completely. Let the model look at **all the words at once**, and let each word decide which other words matter to it. They named the new design the **Transformer**.

> **One-line summary:** RNNs read a book through a keyhole. Transformers lay all the pages on the table and look at everything together.

The original paper tested the Transformer on machine translation (English to German and English to French) and it beat the best earlier results while training faster, because the work could run in parallel.

2. The Transformer: The Big Picture

Picture a translation office with two teams. The **Encoder team** reads the input sentence and writes detailed notes about what every word means in context. The **Decoder team** reads those notes and produces the output sentence, one word at a time.

This is exactly the original Transformer design: an **encoder** stack and a **decoder** stack.

The original Transformer, in numbers

Item	Base model (Vaswani et al., 2017)
Encoder layers	6
Decoder layers	6
Model size (d_model)	512
Attention heads	8
Feed-forward inner size	2048
Parameters (approx.)	65 million (base), 213 million (big)

What is inside one layer?

Think of each layer as a small workshop with a fixed routine. Every **encoder layer** has two steps:

- **Self-attention:** words talk to each other and share information.
- **Feed-forward network:** each word then thinks privately about what it just heard.

Every **decoder layer** has three steps: masked self-attention (it may only look at words it has already written), **cross-attention** (it looks back at the encoder's notes), and then a feed-forward network.

Why stack layers?

Early layers pick up simple patterns, such as which words belong together. Later layers build bigger ideas, such as who did what to whom. Stacking layers is like going from letters to words to sentences to meaning.

3. Self-Attention: The Heart of It All

The classroom story

Read this sentence: *"The cat sat on the mat because **it** was tired."* What does "it" mean? You instantly know it is the cat, not the mat. Your brain linked "it" back to "cat".

Self-attention teaches the model to do the same thing. For every word, the model asks: **"Which other words should I pay attention to in order to understand myself?"** It then blends in information from those words, giving more weight to the important ones.

Query, Key, Value: the library search

Each word creates three small vectors (lists of numbers) about itself. A good way to remember them is a library search:

- **Query (Q):** what I am looking for, like the question you type into the search bar.
- **Key (K):** the label on each book, describing what it contains.
- **Value (V):** the actual content inside the book.

A word's Query is compared with every word's Key. A strong match gives a high score. The scores decide how much of each word's Value gets mixed into the result. So a word ends up as a smart blend of the words most useful to it.

Now the formula (after the feel)

The paper writes the idea as:

> $\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$
> **Read it as:** compare Queries with Keys, scale the scores so they stay calm, turn them into percentages with softmax, and use those percentages to mix the Values.

Multi-head attention: many pairs of eyes

One attention look might focus on grammar, another on meaning, another on nearby words. The Transformer therefore runs **several attention "heads" in parallel** (8 in the base model) and combines what they find. It is like asking eight classmates to highlight the same page, each with a different colour for a different purpose.

4. The Supporting Parts

Positional encoding: remembering the order

Since the Transformer looks at all words at once, it would not know which word came first. "Dog bites man" and "Man bites dog" would look identical. To fix this, the model adds a **position signal** to every word, like numbering the seats in a cinema. The original paper used fixed sine and cosine patterns for this. Later models, including BERT and GPT, learn their position information during training.

Feed-forward network: private thinking time

After words exchange information through attention, each word passes through a small neural network by itself. Attention is the group discussion; the feed-forward network is each student thinking quietly about what they heard.

Residual connections and layer normalisation: safety rails

Deep stacks are hard to train. Two tricks help:

- **Residual connection:** a shortcut that adds the layer's input back to its output, so the original information is never lost.
- **Layer normalisation:** keeps the numbers in a healthy range so training stays stable.

Tokens and embeddings: turning words into numbers

Models cannot read letters. Text is first cut into small pieces called **tokens** (whole words or word-parts), and each token is mapped to a vector called an **embedding**. Words with similar meaning end up with similar embeddings, like friends sitting close together on a map of meaning.

> **Putting it together:** tokens -> embeddings + position signal -> [attention -> feed-forward] repeated many times -> output.

5. BERT: The Reader

BERT stands for **Bidirectional Encoder Representations from Transformers**. Google researchers (Devlin et al.) released it in 2018, and it was presented at NAACL 2019.

The story

Imagine a student who learns English by doing **fill-in-the-blank puzzles**: "The cat sat on the ____." To guess well, the student must read the words on **both sides** of the blank. That is BERT. It is a champion *reader*: it studies a whole sentence in both directions to understand it deeply.

Architecture

BERT uses **only the encoder** half of the Transformer. There is no decoder, so BERT is not built to write long text; it is built to *understand* text.

	BERT-Base	BERT-Large
Layers	12	24
Hidden size	768	1024
Attention heads	12	16
Parameters (approx.)	110 million	340 million

How BERT was trained (pre-training)

- **Masked Language Modeling (MLM):** about 15% of the tokens are hidden, and BERT must guess them from the surrounding words. This is the fill-in-the-blank game.
- **Next Sentence Prediction (NSP):** given two sentences, BERT predicts whether the second really followed the first in the original text.
- **Data:** BooksCorpus (about 800 million words) and English Wikipedia (about 2,500 million words).

Special tokens

[CLS] is placed at the start of the input, and its final representation is used as a summary of the whole sentence for classification tasks. **[SEP]** separates two sentences. **[MASK]** marks a hidden word.

Fine-tuning: from general student to specialist

After pre-training, BERT already knows language in general. To use it for a job, such as sentiment analysis, question answering, or named-entity recognition, you add a small output layer and train a little more on a smaller labelled dataset. This "pre-train once, fine-tune many times" recipe set new records on several benchmarks when BERT was released.

6. GPT: The Writer

GPT stands for **Generative Pre-trained Transformer**. OpenAI introduced the first GPT in 2018 (Radford et al.).

The story

Imagine a storyteller who learns by playing one game over and over: **"Guess the next word."** "Once upon a ____." By playing this game on a huge amount of text, the storyteller learns grammar, facts, and even some reasoning, because to guess the next word well you must understand what is going on. That is GPT: a *writer* that continues text, one token at a time.

Architecture

GPT uses **only the decoder** half of the Transformer (without the cross-attention part, since there is no encoder). The key rule is **causal masking**: when predicting a word, the model can see only the words *before* it, never the future. It is like reading with a card covering everything to the right of your current position.

The GPT family at a glance

Model	Year	Key facts
GPT-1	2018	12-layer decoder, about 117 million parameters. Pre-trained on BooksCorpus, then fine-tuned for each task.
GPT-2	2019	Up to 1.5 billion parameters, trained on WebText (about 40 GB of web text). Showed that a language model can do some tasks with no task-specific training (zero-shot).
GPT-3	2020	175 billion parameters, 96 layers, context window of 2,048 tokens. Showed strong few-shot learning: give a few examples in the prompt and the model picks up the task, with no weight updates.

In-context learning

The big surprise of GPT-3 was that you could *teach it inside the prompt*. Show it two examples of English-to-French translation, then a third English sentence, and it translates it. The model's weights do not change; it simply notices the pattern in the text you gave it. This is called **in-context learning** (few-shot prompting).

From raw GPT to a helpful assistant

A model trained only to predict the next word does not always follow instructions well. Later work, such as InstructGPT (Ouyang et al., 2022), added **instruction tuning** and **reinforcement learning from human feedback (RLHF)** to make models follow what people actually ask. Modern chat assistants build on this idea.

7. BERT vs GPT Side by Side

Both are built from Transformer blocks, but they use different halves of the original design and learn with different games.

Feature	BERT	GPT
Nickname	The reader	The writer
Transformer part used	Encoder only	Decoder only
Direction of reading	Bidirectional (sees left and right)	Left-to-right (sees only the past)
Training game	Fill in masked words (plus next-sentence prediction)	Predict the next word
Best at	Understanding: classification, search, question answering over a given text, named-entity recognition	Generating: writing, chatting, summarising, code, few-shot tasks
Typical output	A representation (embedding) of the text	New text, one token at a time
First released	2018 (Google)	2018 (OpenAI)

A third family: encoder + decoder

The original Transformer used both halves, and some models still do. **T5** (Raffel et al., 2020) treats every task as "text in, text out" using a full encoder-decoder design. Rule of thumb: **encoder-only** for understanding, **decoder-only** for generating, **encoder-decoder** for transforming one text into another, such as translation.

> **Quick memory trick:** BERT is the student who reads the whole page before answering. GPT is the storyteller who keeps talking, one word after another.

8. Why This Matters for RAG

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. (2020). The idea: before the model answers, first *fetch* relevant passages from your own documents, then let a generator write an answer using them. Both model families from this guide appear in a typical RAG app:

- **Reader-style (encoder) models** turn your text chunks and your question into embeddings, so similar meanings can be matched. Sentence-BERT (Reimers and Gurevych, 2019) adapted BERT for exactly this kind of sentence embedding.
- **Writer-style (decoder) models**, the GPT family and its relatives, read the retrieved chunks and write the final answer.

So when you build a PDF chatbot: the **embedding model** plays the reader who files and finds note cards, and the **LLM** plays the writer who composes the answer from those cards.

Glossary

Term	Plain meaning
Token	A small piece of text (a word or part of a word) that the model reads.
Embedding	A list of numbers that captures the meaning of a token or text.
Attention	A way for each word to look at other words and decide which matter.
Self-attention	Attention where a sentence looks at itself.
Encoder	The part that reads and understands input.
Decoder	The part that produces output step by step.
Pre-training	First, broad learning on a huge amount of text.
Fine-tuning	Extra, smaller training to specialise for one job.
Parameters	The learned numbers inside a model; more parameters means a bigger model.
Context window	The maximum amount of text the model can look at at once.

References

Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30 (NeurIPS).

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019 (arXiv:1810.04805).

Radford, A., Narasimhan, K., Salimans, T., Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI.

Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI.

Brown, T., et al. (2020). Language Models are Few-Shot Learners. NeurIPS 33 (arXiv:2005.14165).

Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS 35.

Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR 21.

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 33.

Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP 2019.